

**ĐẠI HỌC HUẾ
TRƯỜNG ĐẠI HỌC KHOA HỌC**



TIỂU LUẬN

HỌC PHẦN: PHÁT TRIỂN ỨNG DỤNG IOT

ĐỀ TÀI:

Điều khiển từ xa qua giao thức MQTT với ESP32

**Giảng viên hướng dẫn : Võ Việt Dũng
Sinh viên thực hiện: Đặng Hữu Trung
Mã sinh viên: 22T1020484**

1. Danh mục các từ viết tắt

- **ESP:** Espressif Systems (Hãng sản xuất vi điều khiển)
- **HTTP:** HyperText Transfer Protocol (Giao thức truyền tải siêu văn bản)
- **IoT:** Internet of Things (Internet vạn vật)
- **MQTT:** Message Queuing Telemetry Transport (Giao thức truyền tin nhắn từ xa)
- **QoS:** Quality of Service (Chất lượng dịch vụ)

2. Mục lục

1. Danh mục các từ viết tắt	2
2. Mục lục	3
3. Phần Mở đầu	1
4. Phần Nội dung.....	1
4.1. Cơ sở lý thuyết của hệ thống	1
4.2. Thiết kế hệ thống điều khiển và giám sát.....	1
4.3. Kết quả mô phỏng trên nền tảng Wokwi và ThingSpeak	2
5. Phần Kết luận/Nhận xét/Đánh giá/Kiến nghị	3
6. Tài liệu tham khảo	1
7. Phụ lục	1

3. Phần Mở đầu

Sự bùng nổ của Internet vạn vật (IoT) đã thay đổi hoàn toàn cách con người tương tác với các thiết bị điện tử xung quanh. Nhu cầu giám sát môi trường và điều khiển thiết bị từ xa không chỉ dừng lại ở các ứng dụng công nghiệp mà đã đi sâu vào đời sống hàng ngày thông qua các mô hình nhà thông minh (Smart Home).

Mục tiêu của tiểu luận này là thiết kế và mô phỏng một hệ thống điều khiển bật/tắt thiết bị từ xa, kết hợp giám sát thông số môi trường. Hệ thống sử dụng vi điều khiển ESP32 làm trung tâm xử lý, giao thức MQTT với broker Mosquitto (được mô phỏng qua HiveMQ) để nhận lệnh điều khiển thời gian thực, và nền tảng ThingSpeak để lưu trữ, trực quan hóa dữ liệu. Việc ứng dụng Wokwi để mô phỏng không chỉ giúp tiết kiệm chi phí phần cứng mà còn minh chứng tính khả thi của thuật toán trước khi triển khai thực tế.

4. Phần Nội dung

4.1. Cơ sở lý thuyết của hệ thống

Giao thức MQTT là nền tảng cốt lõi cho tính năng điều khiển thời gian thực của hệ thống. Đây là một giao thức nhắn tin dạng xuất bản/đăng ký (publish/subscribe), được thiết kế đặc biệt cho các thiết bị bị hạn chế về tài nguyên và kết nối mạng băng thông thấp [1, 46]. MQTT hoạt động dựa trên một máy chủ trung tâm gọi là Broker. Trong phạm vi nghiên cứu này, ESP32 đóng vai trò là một Client (Subscriber) luôn lắng nghe một chủ đề (Topic) nhất định từ Broker để kích hoạt thiết bị.

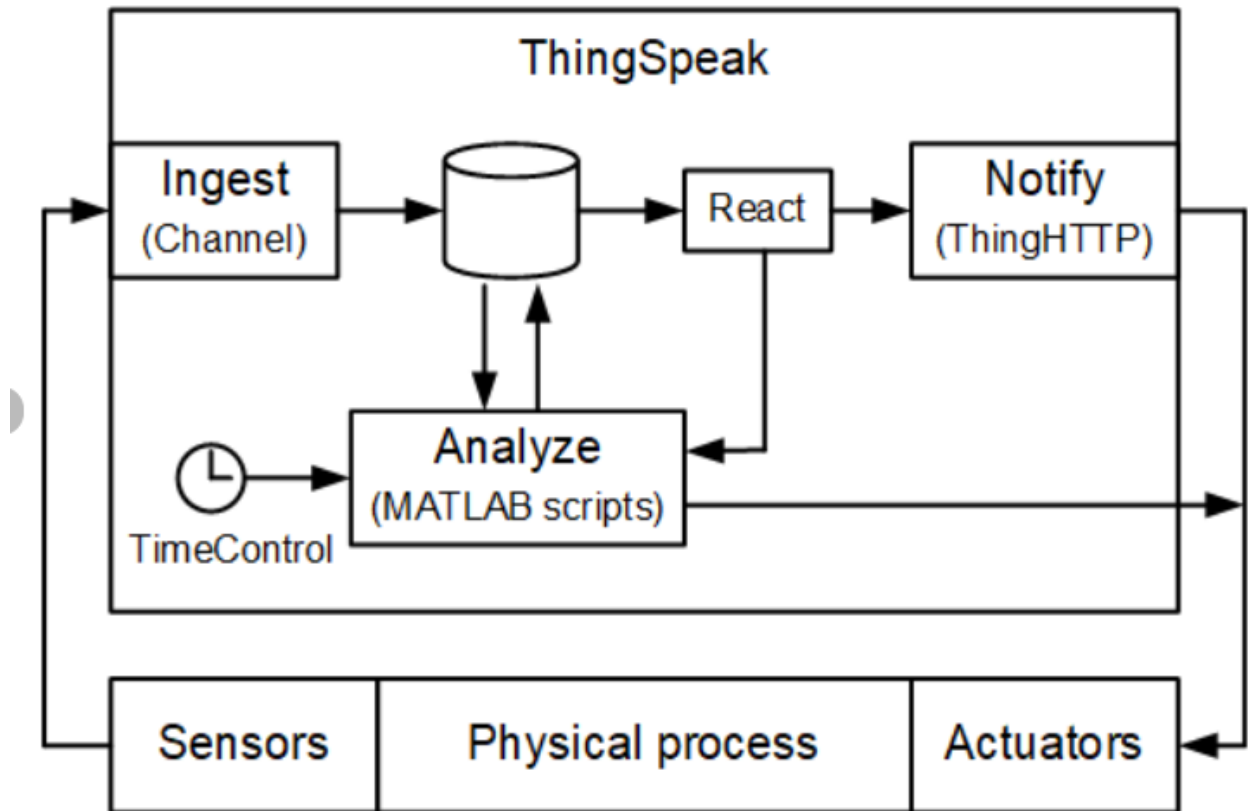
Bên cạnh đó, quá trình đẩy dữ liệu cảm biến lên đám mây được thực hiện qua giao thức HTTP (phương thức GET/POST). Nền tảng ThingSpeak được lựa chọn nhờ khả năng cung cấp API mở, cho phép thiết bị IoT gửi dữ liệu chuỗi thời gian lên hệ thống máy chủ và tự động vẽ thành các biểu đồ trực quan [2, 115].

4.2. Thiết kế hệ thống điều khiển và giám sát

Hệ thống được thiết kế gồm ba khối chính:

- **Khối xử lý trung tâm:** Sử dụng vi điều khiển ESP32 do có tích hợp sẵn module Wi-Fi, tốc độ xử lý nhanh và hỗ trợ đa dạng các giao thức truyền thông [4].
- **Khối cảm biến và chấp hành:** Bao gồm cảm biến DHT22 để thu thập nhiệt độ, độ ẩm. Các dữ liệu về nồng độ khí gas được lập trình sinh giá trị ngẫu nhiên để mô phỏng một môi trường thu thập đa dữ liệu. Khối chấp hành sử dụng đèn LED làm đại diện cho Relay điều khiển dòng điện 220V.
- **Khối truyền thông và giao diện giám sát:** Sử dụng nền tảng Wokwi để mô phỏng mạch điện vật lý, Cloud ThingSpeak để vẽ biểu đồ và HiveMQ Web Client đóng vai trò như thiết bị di động gửi lệnh điều khiển.

Hình ảnh sơ đồ kiến trúc tổng quát của hệ thống giám sát và điều khiển qua MQTT.

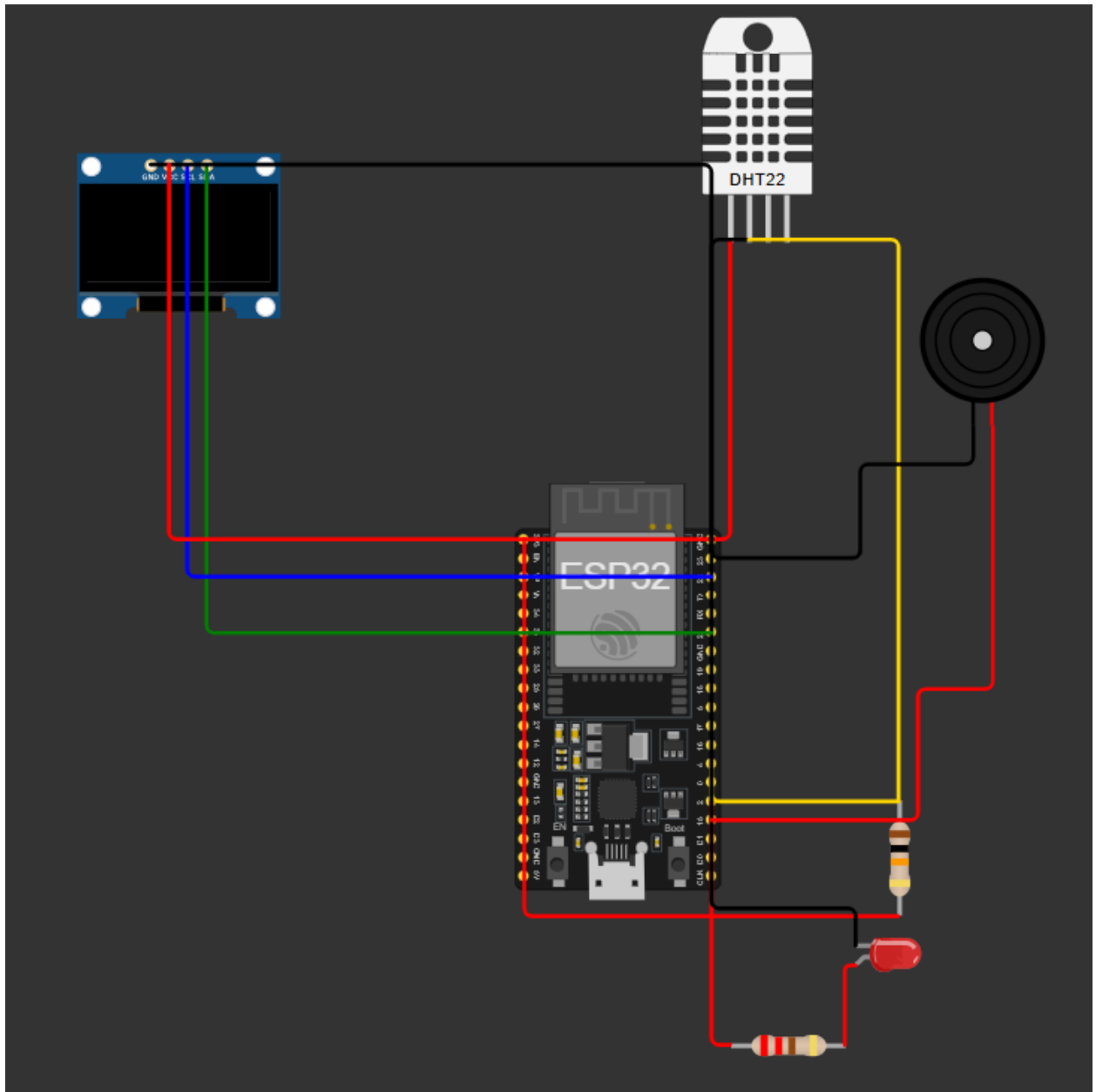


4.3. Kết quả mô phỏng trên nền tảng Wokwi và ThingSpeak

Hệ thống đã được lập trình và lắp ráp trên nền tảng Wokwi. Chu trình hoạt động diễn ra ổn định theo hai luồng song song:

- *Luồng dữ liệu (Data Logging)*: Cứ sau 15 giây, ESP32 thu thập giá trị nhiệt độ, độ ẩm từ DHT22 và giá trị khí gas giả lập. Gói dữ liệu này được đóng gói và gửi qua HTTP Request đến API của ThingSpeak. Kết quả hiển thị trên ThingSpeak cho thấy 3 biểu đồ (Field 1, 2, 3) cập nhật liên tục, đáp ứng yêu cầu giám sát dài hạn.

Hình ảnh sơ đồ kết nối phần cứng mô phỏng trên nền tảng Wokwi



- *Luồng điều khiển (Control):* ESP32 duy trì kết nối liên tục với MQTT Broker qua cổng 1883. Khi người dùng truy cập Web Client và gửi (Publish) chuỗi "ON" hoặc "OFF" vào topic đã định sẵn, ESP32 lập tức nhận được lệnh (độ trễ dưới 1 giây) và thực hiện thay đổi trạng thái của đèn LED [3].

5. Phần Kết luận/Nhận xét/Đánh giá/Kiến nghị

Kết luận và Đánh giá: Hệ thống điều khiển và giám sát thiết bị từ xa qua giao thức MQTT sử dụng ESP32 đã hoạt động đúng với thiết kế ban đầu. Quá trình mô phỏng trên Wokwi diễn ra thành công: hệ thống cập nhật chính xác các thông số môi trường lên

ThingSpeak và phản hồi tức thời với các lệnh bật/tắt từ MQTT Broker. Giải pháp này cho thấy tính ổn định, tiết kiệm tài nguyên mạng và dễ dàng mở rộng số lượng node cảm biến.

Nhận xét và Kiến nghị: Mặc dù hệ thống mô phỏng hoạt động tốt, thực tế triển khai cần tính toán đến nhiều sóng Wi-Fi và sự sụt áp của phần cứng. Để mở rộng hướng nghiên cứu, có thể phát triển thêm các ứng dụng Web (Web Application Development) đóng vai trò làm Dashboard tùy biến thay vì phụ thuộc hoàn toàn vào giao diện có sẵn của ThingSpeak hay MQTT Web Client, giúp tối ưu hóa trải nghiệm người dùng cuối.

6. Tài liệu tham khảo

Tiếng Việt: [1] Trần Thuận Hoàng (2021). Ứng dụng IoT trong điều khiển thông minh, Tạp chí Khoa học và Công nghệ, tập 15, số 3, tr. 45-50. [2] Nguyễn Đình Phú (2019). Giáo trình Vi điều khiển và Ứng dụng, Nxb Đại học Quốc gia TP.HCM, TP.HCM.

Tiếng Anh: [3] O'Leary, N. (2018). PubSubClient - A client library for MQTT. <http://pubsubclient.knolleary.net/>. Truy cập ngày 08/04/2026. [4] Youssef, M. (2020). ESP32 with DHT11/DHT22 Temperature and Humidity Sensor using Arduino IDE. <https://randomnerdtutorials.com/esp32-dht11-dht22-temperature-humidity-sensor-arduino-ide/>. Truy cập ngày 08/04/2026.

7. Phụ lục

Phụ lục 1: Mã nguồn C++ điều khiển vi điều khiển ESP32 tích hợp giao thức MQTT và ThingSpeak

```
#include <WiFi.h>

#include <PubSubClient.h>

#include <HTTPClient.h>

#include "DHT.h"

// --- 1. Cấu hình mạng WiFi ---

const char* ssid = "Wokwi-GUEST"; // Sử dụng mạng ảo của nền tảng Wokwi

const char* password = "";

// --- 2. Cấu hình MQTT Broker (Sử dụng HiveMQ Public Broker) ---
```

```

const char* mqtt_server = "broker.hivemq.com";
const char* mqtt_topic_control = "doan/dieukhien/thietbi1";

// --- 3. Cấu hình Cloud ThingSpeak ---
const char* serverThingSpeak = "http://api.thingspeak.com/update";
String apiKeyThingSpeak = "THAY_BANG_WRITE_API_KEY_CUA_BAN";

// --- 4. Cấu hình Cảm biến & Chân điều khiển thiết bị ---
#define DHTPIN 15
#define DHTTYPE DHT22
DHT dht(DHTPIN, DHTTYPE);

const int relayPin = 2; // Chân GPIO kết nối với Relay/LED

// Khởi tạo đối tượng mạng và giao thức
WiFiClient espClient;
PubSubClient mqttClient(espClient);

// Biến quản lý thời gian gửi dữ liệu (chu kỳ 15 giây)
unsigned long lastSendTime = 0;
const long sendInterval = 15000;

void setup() {
    Serial.begin(115200);

    // Thiết lập chân điều khiển Relay
    pinMode(relayPin, OUTPUT);
    digitalWrite(relayPin, LOW); // Trạng thái ban đầu: Tắt thiết bị

    dht.begin();

```



```

// Khởi tạo kết nối WiFi
Serial.print("Đang kết nối WiFi");
WiFi.begin(ssid, password);
while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
}
Serial.println("\nĐã kết nối WiFi thành công!");

// Thiết lập thông số MQTT Broker
mqttClient.setServer(mqtt_server, 1883);
mqttClient.setCallback(mqttCallback);
}

// Hàm Callback xử lý tin nhắn nhận được từ MQTT Broker
void mqttCallback(char* topic, byte* payload, unsigned int length) {
    String message = "";
    for (int i = 0; i < length; i++) {
        message += (char)payload[i];
    }

    Serial.print("Nhận được lệnh từ MQTT: ");
    Serial.println(message);

    // Xử lý logic bật/tắt dựa trên nội dung tin nhắn
    if (message == "ON") {
        digitalWrite(relayPin, HIGH);
        Serial.println("-> TRẠNG THÁI: THIẾT BỊ ĐÃ BẬT");
    } else if (message == "OFF") {

```

```

        digitalWrite(relayPin, LOW);
        Serial.println("-> TRẠNG THÁI: THIẾT BỊ ĐÃ TẮT");
    }
}

// Hàm kiểm tra và duy trì kết nối với MQTT Broker
void reconnectMQTT() {
    while (!mqttClient.connected()) {
        Serial.print("Đang kết nối lại với MQTT Broker...");
        // Tạo Client ID ngẫu nhiên để tránh xung đột
        String clientId = "ESP32Client-" + String(random(0xffff), HEX);

        if (mqttClient.connect(clientId.c_str())) {
            Serial.println(" Thành công!");
            mqttClient.subscribe(mqtt_topic_control); // Đăng ký lại topic sau khi
            kết nối
        } else {
            Serial.print(" Thất bại, mã lỗi rc=");
            Serial.print(mqttClient.state());
            Serial.println(" Thử lại sau 5 giây...");
            delay(5000);
        }
    }
}

// Hàm đóng gói và gửi dữ liệu qua giao thức HTTP (ThingSpeak API)
void sendDataToThingSpeak(float t, float h, int gas) {
    if (WiFi.status() == WL_CONNECTED) {
        HTTPClient http;
        // Xây dựng chuỗi URL chứa các tham số (Field 1, 2, 3)

```

```

String url = String(serverThingSpeak) + "?api_key=" + apiKeyThingSpeak +
            "&field1=" + String(t) +
            "&field2=" + String(h) +
            "&field3=" + String(gas);

http.begin(url);
int httpCode = http.GET(); // Thực hiện phương thức GET

if (httpCode > 0) {
    Serial.println("Đã đồng bộ dữ liệu lên ThingSpeak thành công!");
} else {
    Serial.println("Lỗi đồng bộ dữ liệu lên ThingSpeak.");
}
http.end(); // Giải phóng tài nguyên
}
}

void loop() {
    // Đảm bảo kết nối MQTT luôn được duy trì
    if (!mqttClient.connected()) {
        reconnectMQTT();
    }
    mqttClient.loop(); // Vòng lặp xử lý các gói tin MQTT đến

    unsigned long currentMillis = millis();

    // Khởi lệnh thực thi định kỳ (Thu thập và gửi dữ liệu cảm biến)
    if (currentMillis - lastSendTime >= sendInterval) {
        lastSendTime = currentMillis;
    }
}

```

```

// Đọc giá trị thực tế từ cảm biến DHT22
float h = dht.readHumidity();
float t = dht.readTemperature();

// Sinh giá trị ảo mô phỏng cảm biến nồng độ khí Gas (phạm vi 300 - 800)
int randomGas = random(300, 800);

// Kiểm tra tính hợp lệ của dữ liệu đầu vào
if (isnan(h) || isnan(t)) {
    Serial.println("Lỗi: Không thể đọc dữ liệu từ cảm biến DHT!");
    return;
}

// Xuất dữ liệu ra Serial Monitor để giám sát cục bộ
Serial.printf("Nhiệt độ: %.1f°C | Độ ẩm: %.1f%% | Khí Gas giả lập: %d\n",
t, h, randomGas);

// Đẩy dữ liệu đã thu thập lên Cloud
sendDataToThingSpeak(t, h, randomGas);
}
}

```

Phụ lục 2: Bảng chi tiết đấu nối chân linh kiện (Pinout/Wiring)

Bảng dưới đây trình bày chi tiết sơ đồ đấu nối giữa vi điều khiển trung tâm ESP32 và các module ngoại vi trong hệ thống mô phỏng/thực tế:

STT	Thiết bị ngoại vi	Chân linh kiện	Kết nối với chân ESP32	Ghi chú
1	Cảm biến DHT22	VCC	3V3 / VIN	Cấp nguồn 3.3V hoặc 5V
2	Cảm biến DHT22	GND	GND	Nối đất
3	Cảm biến DHT22	DATA	GPIO 15	Truyền nhận dữ liệu cảm biến
4	Module Relay / LED	VCC / Anode	GPIO 2	Nhận tín hiệu điều khiển (High/Low)
5	Module Relay / LED	GND / Cathode	GND	Nối qua điện trở 330Ω (nếu là LED)

Phụ lục 3: Danh sách các thư viện phần mềm (Libraries) được sử dụng

Để mã nguồn trong Phụ lục 1 có thể biên dịch và hoạt động thành công trên Arduino IDE hoặc Wokwi, hệ thống yêu cầu cài đặt các thư viện sau:

1. **WiFi.h**: Thư viện tiêu chuẩn của ESP32 để quản lý kết nối mạng Wi-Fi.
2. **HTTPClient.h**: Thư viện tiêu chuẩn hỗ trợ tạo các yêu cầu HTTP (GET/POST) để gửi dữ liệu lên ThingSpeak API.
3. **PubSubClient (tác giả Nick O'Leary) - Phiên bản 2.8**: Thư viện hỗ trợ thiết lập MQTT Client, cho phép ESP32 Publish và Subscribe các topic từ Broker.
4. **DHT sensor library (tác giả Adafruit) - Phiên bản 1.4.4**: Thư viện hỗ trợ đọc dữ liệu nhiệt độ, độ ẩm từ các dòng cảm biến DHT (DHT11, DHT22).
5. **Adafruit Unified Sensor - Phiên bản 1.1.9**: Thư viện nền tảng bắt buộc đi kèm để thư viện DHT hoạt động.

PHIẾU ĐÁNH GIÁ TIỂU LUẬN

Học kỳ Năm học ...-...

Cán bộ chấm thi 1	Cán bộ chấm thi 2
Nhận xét:	Nhận xét:
.....	
.....	
.....	
.....	
.....	
.....	
.....	
.....	
.....	
.....	
.....	
Điểm đánh giá của CBChT1:	Điểm đánh giá của CBChT2:
Bằng số:	Bằng số:
Bằng chữ:	Bằng chữ:

Điểm kết luận: Bằng số..... Bằng chữ:.....

Thừa Thiên Huế, ngày tháng năm 20...

CBChT1

(Ký và ghi rõ họ tên)

CBChT2

(Ký và ghi rõ họ tên)